

Top 20 RxJS Interview Questions and Answers for Senior Angular Developers

Senior-level interview preparation

Angular + RxJS + TypeScript

These answers are designed for experienced Angular developers preparing for technical interviews, including Hexaware. Each question includes a practical explanation, implementation example, and important interview considerations.

The focus is on **real-world scenarios, performance, memory management, API integration, caching, and production-ready architecture.**

1. How do you implement HTTP response caching in Angular using RxJS?

Hexaware interview question

Interview answer

Angular's `HttpClient` returns cold Observables. Every independent subscription to an HTTP Observable normally triggers a new HTTP request.

We can use RxJS `shareReplay(1)` to share a single HTTP execution among subscribers and cache its latest successful response.

Example: Employee service with caching

TypeScript

```
import { Injectable, inject } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable, shareReplay } from 'rxjs';

interface Employee {
  id: number;
  name: string;
}

@Injectable({ providedIn: 'root' })
export class EmployeeService {
  private http = inject(HttpClient);

  private employees$?: Observable<Employee[]>;
```

```

getEmployees(): Observable<Employee[]> {
  if (!this.employees$) {
    this.employees$ = this.http
      .get<Employee[]>('/api/employees')
      .pipe(
        shareReplay({
          bufferSize: 1,
          refCount: false
        })
      );
  }

  return this.employees$;
}

```

Component usage

TypeScript

```
employees$ = inject(EmployeeService).getEmployees();
```

HTML

```

@if (employees$ | async; as employees) {
  @for (employee of employees; track employee.id) {
    <p>{{ employee.name }}</p>
  }
}

```

How it works

Component A — subscribes

Component B — subscribes



EmployeeService

Cached Observable + `shareReplay(1)`



HTTP GET /api/employees

Senior-level considerations:

- Cache Observables in a singleton service rather than creating a new `shareReplay` pipeline for every method call.
- Define cache invalidation rules when data changes.
- Avoid caching sensitive data indefinitely.
- For long-lived streams, consider `refCount: true` and explicit lifecycle management.
- A completed HTTP request with `refCount: false` can retain its cached response for later subscribers.

Interview tip: `shareReplay(1)` alone does not create a service-wide cache. The shared Observable must be retained and reused.

2. How does `shareReplay(1)` prevent duplicate HTTP requests and asynchronous race conditions?

Interview answer

`shareReplay(1)` multicasts one source subscription and replays the latest emitted value to new subscribers.

It prevents duplicate HTTP requests when multiple consumers subscribe to the **same shared Observable instance**.

Without `shareReplay`

TypeScript

```
const employees$ =
  this.http.get('/api/employees');

employees$.subscribe();
employees$.subscribe();

// Two independent HTTP requests
```

With `shareReplay`

TypeScript

```
const employees$ = this.http
  .get('/api/employees')
  .pipe(shareReplay(1));

employees$.subscribe();
employees$.subscribe();

// One HTTP request through this shared instance
```

What happens internally?

- T1 Subscriber A subscribes.
- T2 HTTP request starts.
- T3 Subscriber B joins the same request.
- T4 Response arrives and is delivered to both.
- T5 Subscriber C receives the cached response.

Does shareReplay prevent all race conditions?

No. It prevents duplicate execution and ensures subscribers to one shared stream observe the same response.

It does not prevent stale results from separate requests overwriting newer results.

For example, when users rapidly change a search term, use `switchMap` to cancel the previous request.

TypeScript

```
results$ = searchTerm$.pipe(
  switchMap(term =>
    this.http.get('/api/search', {
      params: { q: term }
    })
  )
);
```

Interview tip: Use `shareReplay` for request sharing and `switchMap` for latest-request-wins behavior. They solve different problems.

3. How would you implement cache clearing and force-refresh functionality using RxJS?

Interview answer

A production application needs an explicit cache invalidation strategy.

We can implement a service that stores a shared HTTP Observable and clears its reference when fresh data is required.

TypeScript

```
@Injectable({ providedIn: 'root' })
export class EmployeeService {
  private http = inject(HttpClient);

  private cache$: Observable<Employee[]>;

  getEmployees(): Observable<Employee[]> {
    if (!this.cache$) {
      this.cache$ = this.http
        .get<Employee[]>('/api/employees')
        .pipe(
          shareReplay(1)
        );
    }

    return this.cache$;
  }

  clearCache(): void {
    this.cache$ = undefined;
  }

  refresh(): Observable<Employee[]> {
    this.clearCache();
    return this.getEmployees();
  }
}
```

Usage

TypeScript

```
// Initial request
this.employeeService.getEmployees();

// Clear cache
```

```
this.employeeService.clearCache();

// Fetch fresh data
this.employeeService.refresh()
  .subscribe(data => console.log(data));
```

Important production consideration

Clearing the cached Observable reference does not automatically update existing subscribers or cancel requests already in progress.

For reactive refresh, use a refresh trigger.

TypeScript

```
private refreshTrigger$ =
  new BehaviorSubject<void>(undefined);

employees$ = this.refreshTrigger$.pipe(
  switchMap(() =>
    this.http.get<Employee[]>('/api/employees')
  ),
  shareReplay({
    bufferSize: 1,
    refCount: true
  })
);

refresh(): void {
  this.refreshTrigger$.next();
}
```

Here, every refresh triggers a new request, and `switchMap` cancels the previous request when a newer refresh arrives.

Interview tip: Distinguish between invalidating a cached reference, triggering a new request, and updating existing subscribers. They are separate operations.

4. How do you share state between unrelated Angular components using BehaviorSubject?

Interview answer

`BehaviorSubject` is useful for sharing application state because it stores the current value and immediately emits that value to new subscribers.

For example, a header component and an employee component may need to access the same selected employee without having a parent-child relationship.

Shared state service

TypeScript

```
@Injectable({ providedIn: 'root' })
export class EmployeeStateService {

  private selectedEmployeeSubject =
    new BehaviorSubject<Employee | null>(null);

  readonly selectedEmployee$ =
    this.selectedEmployeeSubject.asObservable();

  selectEmployee(employee: Employee): void {
    this.selectedEmployeeSubject.next(employee);
  }

  clearSelection(): void {
    this.selectedEmployeeSubject.next(null);
  }

  getCurrentEmployee(): Employee | null {
    return this.selectedEmployeeSubject.value;
  }
}
```

Component A: Update state

TypeScript

```
export class EmployeeListComponent {
  private state = inject(EmployeeStateService);

  select(employee: Employee): void {
    this.state.selectEmployee(employee);
  }
}
```

Component B: Read state

TypeScript

```
export class HeaderComponent {
  private state = inject(EmployeeStateService);
```

```
selectedEmployee$ = this.state.selectedEmployee$;  
}
```

HTML

```
@if (selectedEmployee$ | async; as employee) {  
  <p>Selected: {{ employee.name }}</p>  
}
```

Senior-level considerations: Keep the Subject private, expose a read-only Observable, avoid mutating shared objects directly, and use a service scope appropriate to the state. For complex applications, consider Angular Signals or a dedicated state-management solution.

5. What is the difference between switchMap, mergeMap, concatMap and exhaustMap?

Interview answer

All four are higher-order mapping operators. They transform source emissions into inner Observables and determine how those inner Observables are subscribed to.

Comparison

Operator	Behavior	Practical use
switchMap	Cancels previous inner subscription	Search, latest data
mergeMap	Runs inner subscriptions concurrently	Independent API calls
concatMap	Queues and executes sequentially	Ordered updates
exhaustMap	Ignores new emissions while busy	Prevent duplicate submissions

Visual example

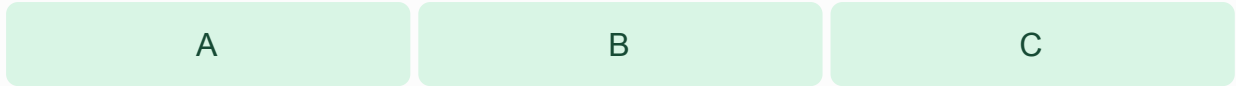
Assume a user triggers A, B and C quickly.

switchMap



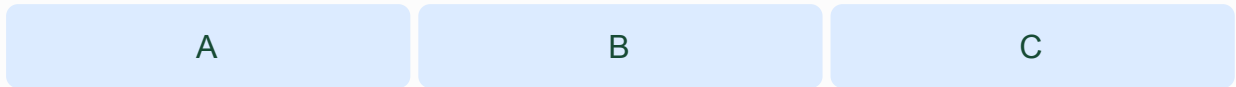
Previous operation is cancelled

mergeMap



All operations may run concurrently

concatMap



Operations execute in order

exhaustMap



New emissions are ignored while A is active

Illustrative behavior when A, B and C arrive before the first operation completes.

Example 1: switchMap for search

TypeScript

```
searchResults$ = searchControl.valueChanges.pipe(  
  debounceTime(300),  
  distinctUntilChanged(),  
  switchMap(term =>  
    this.http.get<Employee[]>('/api/search', {  
      params: { q: term }  
    })  
  )  
);
```

Example 2: mergeMap for independent requests

TypeScript

```
from([1, 2, 3]).pipe(  
  mergeMap(id =>  
    this.http.get(`/api/employees/${id}`),  
    3 // Maximum concurrency  
  )  
).subscribe();
```

Example 3: concatMap for ordered updates

TypeScript

```

saveRequests$.pipe(
  concatMap(employee =>
    this.http.put(
      `/api/employees/${employee.id}`,
      employee
    )
  )
).subscribe();

```

Example 4: exhaustMap for login

TypeScript

```

loginClicks$.pipe(
  exhaustMap(credentials =>
    this.authService.login(credentials)
  )
).subscribe();

```

Senior interview trap: `switchMap` cancels the client subscription, but cancellation does not guarantee that a server has stopped processing a request. Avoid assuming that it can roll back a database update.

6. What is the difference between Observable, Subject, BehaviorSubject, ReplaySubject and AsyncSubject?

Interview answer

An Observable represents a stream of values. Subjects are both Observables and observers, allowing values to be pushed to multiple subscribers.

Type	Initial value	Replays values	Common use
Observable	Not required	Depends on implementation	HTTP, events
Subject	No	No	Event broadcasting
BehaviorSubject	Required	Latest value	Shared state
ReplaySubject	No	Configured previous values	Event history
AsyncSubject	No	Final value after completion	Final result

Observable

TypeScript

```
const data$ = new Observable<number>(  
  observer => {  
    observer.next(10);  
    observer.next(20);  
    observer.complete();  
  }  
);  
  
data$.subscribe(console.log);  
// 10, 20
```

Subject

TypeScript

```
const subject = new Subject<number>();  
  
subject.next(10); // No subscriber yet  
  
subject.subscribe(console.log);  
  
subject.next(20); // Output: 20
```

BehaviorSubject

TypeScript

```
const state$ = new BehaviorSubject<number>(0);  
  
state$.next(10);  
  
state$.subscribe(console.log);  
// Output: 10
```

ReplaySubject

TypeScript

```
const history$ = new ReplaySubject<number>(2);  
  
history$.next(10);  
history$.next(20);  
history$.next(30);
```

```
history$.subscribe(console.log);  
// Output: 20, 30
```

AsyncSubject

TypeScript

```
const result$ = new AsyncSubject<number>();  
  
result$.subscribe(console.log);  
  
result$.next(10);  
result$.next(20);  
  
result$.complete();  
  
// Output: 20
```

Interview tip: A `BehaviorSubject` always has a current value, whereas a `ReplaySubject(1)` can be created without an initial value. An `AsyncSubject` emits its final value only upon completion.

7. How do you prevent memory leaks in Angular applications using RxJS?

Interview answer

Memory leaks can occur when long-lived subscriptions retain references to components after those components are destroyed.

Angular provides several ways to manage subscriptions safely.

Method 1: AsyncPipe

TypeScript

```
employees$ = this.employeeService.getEmployees();
```

HTML

```
@for (employee of (employees$ | async) ?? [];  
      track employee.id) {  
  <p>{{ employee.name }}</p>  
}
```

`AsyncPipe` automatically unsubscribes when the view is destroyed.

Method 2: takeUntilDestroyed

Recommended for imperative subscriptions in modern Angular.

TypeScript

```
import {
  Component,
  DestroyRef,
  inject
} from '@angular/core';

import { takeUntilDestroyed }
  from '@angular/core/rxjs-interop';

@Component({
  selector: 'app-employee',
  template: ''
})
export class EmployeeComponent {

  private destroyRef = inject(DestroyRef);
  private service = inject(EmployeeService);

  ngOnInit(): void {
    this.service.getEmployees()
      .pipe(
        takeUntilDestroyed(this.destroyRef)
      )
      .subscribe(data => {
        console.log(data);
      });
  }
}
```

Method 3: takeUntil for older applications

TypeScript

```
private destroy$ = new Subject<void>();

ngOnInit(): void {
  interval(1000).pipe(
    takeUntil(this.destroy$)
  ).subscribe(console.log);
}
```

```
ngOnDestroy(): void {  
  this.destroy$.next();  
  this.destroy$.complete();  
}
```

Method 4: take(1)

TypeScript

```
this.employeeService.getEmployees()  
  .pipe(take(1))  
  .subscribe(console.log);
```

`take(1)` automatically completes after the first emission. Use it only when one value is sufficient.

Senior-level considerations:

- HTTP Observables normally complete after a response, but a subscription can still need cancellation when a component is destroyed.
- `interval`, `valueChanges`, `Subjects` and `WebSocket` streams can remain active indefinitely.
- `shareReplay({ bufferSize: 1, refCount: false })` can keep a non-completing source subscribed even after all consumers unsubscribe.
- A subscription should be managed by the component, service or application layer that owns its lifecycle.

8. What is the difference between hot and cold Observables?

Interview answer

A cold Observable normally creates an independent execution for every subscriber.

A hot Observable shares an existing producer, so subscribers may join after values have already been emitted.

Cold Observable

Execution commonly starts on subscription

Each subscriber can get separate execution

Hot Observable

Producer can exist independently

Subscribers share a producer

HTTP requests are usually cold

Subjects and shared event streams are hot

Values are produced per subscription

Late subscribers may miss values

Cold Observable example

TypeScript

```
const data$ = new Observable(observer => {
  console.log('Execution started');
  observer.next(Math.random());
});

data$.subscribe(console.log);
data$.subscribe(console.log);

// Execution starts twice
```

Hot Observable example

TypeScript

```
const subject = new Subject<number>();

subject.subscribe(value =>
  console.log('A:', value)
);

subject.subscribe(value =>
  console.log('B:', value)
);

subject.next(100);

// A: 100
// B: 100
```

Interview tip: Hot and cold describe producer and subscription behavior, not simply whether an Observable is asynchronous. Operators such as `share()` can turn an independently executed source into a shared stream.

9. What is the difference between `share()` and `shareReplay()`?

Interview answer

Both operators multicast a source Observable so multiple subscribers can share one execution.

The main difference is that `shareReplay()` also buffers values for late subscribers.

Feature	<code>share()</code>	<code>shareReplay(1)</code>
Shares source subscription	Yes	Yes
Replays previous values	No	Yes
Late subscribers receive cached value	No	Yes
Useful for	Live events	HTTP caching, shared data
Default reference counting	Yes	No

`share()`

TypeScript

```
const shared$ = interval(1000).pipe(
  share()
);

shared$.subscribe(value =>
  console.log('A:', value)
);

setTimeout(() => {
  shared$.subscribe(value =>
    console.log('B:', value)
  );
}, 3000);
```

Subscriber B joins the ongoing stream without automatically receiving earlier values.

`shareReplay()`

TypeScript

```
const shared$ = interval(1000).pipe(
  shareReplay({
    bufferSize: 1,
    refCount: true
  })
);
```

```
    })  
  );  
}
```

Production tip: For a continuous source, `refCount: true` allows the upstream subscription to be released when there are no subscribers. For a one-shot HTTP cache, `refCount: false` can be appropriate when retaining an in-flight request and its completed response is intentional.

10. How would you implement autocomplete search using RxJS?

Interview answer

Autocomplete should avoid unnecessary API calls, cancel outdated requests, handle errors and display only relevant results.

The typical operator combination is:

```
debounceTime → distinctUntilChanged → switchMap → catchError
```

Angular implementation

TypeScript

```
import { FormControl } from '@angular/forms';  
import {  
  debounceTime,  
  distinctUntilChanged,  
  switchMap,  
  catchError,  
  of,  
  startWith  
} from 'rxjs';  
  
searchControl = new FormControl('', {  
  nullable: true  
});  
  
results$ = this.searchControl.valueChanges.pipe(  
  startWith(this.searchControl.value),  
  
  debounceTime(300),  
  
  map(term => term.trim()),  
  
  distinctUntilChanged(),  

```

```

switchMap(term => {
  if (term.length < 2) {
    return of([]);
  }

  return this.http.get<Employee[]>(
    '/api/employees/search',
    { params: { q: term } }
  ).pipe(
    catchError(() => of([]))
  );
})
);

```

Add `map` to the RxJS imports in this example.

Template

HTML

```

<input
  type="text"
  [formControl]="searchControl"
  placeholder="Search employees"
/>

@for (employee of (results$ | async) ?? [];
  track employee.id) {
  <div>{{ employee.name }}</div>
}

```

Why these operators?

Operator	Purpose
<code>debounceTime(300)</code>	Wait until typing pauses
<code>map()</code>	Normalize search input
<code>distinctUntilChanged()</code>	Skip consecutive duplicate terms
<code>switchMap()</code>	Cancel obsolete requests
<code>catchError()</code>	Recover from request failures

Senior interview trap: Place `catchError` inside `switchMap` when you want the search stream to survive individual HTTP failures. An outer `catchError` returning `of([])`

completes the overall search stream after an error.

11. What is the difference between combineLatest, forkJoin, zip and withLatestFrom?

Interview answer

These operators combine multiple Observables, but differ in when they emit and how they pair values.

Operator	Emission behavior	Use case
<code>combineLatest</code>	Emits after all sources have emitted, then on any source update	Reactive dashboards
<code>forkJoin</code>	Emits once after all sources complete	Parallel HTTP calls
<code>zip</code>	Pairs emissions by index	Coordinated sequences
<code>withLatestFrom</code>	Emits when the primary source emits, using latest secondary values	Combine click with state

combineLatest

TypeScript

```
combineLatest([
  this.employeeService.getEmployees(),
  this.departmentService.getDepartments()
]).subscribe(([employees, departments]) => {
  console.log(employees, departments);
});
```

It waits for both sources to emit at least once. Subsequent emissions from either source produce a combined result.

forkJoin

TypeScript

```
forkJoin({
  employees: this.http.get('/api/employees'),
  departments: this.http.get('/api/departments')
}).subscribe(result => {
  console.log(result.employees);
});
```

```
console.log(result.departments);  
});
```

Both HTTP requests execute concurrently. `forkJoin` emits their final values when both complete.

zip

TypeScript

```
zip(  
  of(1, 2, 3),  
  of('A', 'B', 'C')  
) .subscribe(console.log);  
  
// [1, 'A']  
// [2, 'B']  
// [3, 'C']
```

withLatestFrom

TypeScript

```
saveClick$.pipe(  
  withLatestFrom(formValue$)  
) .subscribe(([click, formValue]) => {  
  console.log(formValue);  
});
```

It emits only when `saveClick$` emits, provided `formValue$` has emitted at least once.

Senior interview trap: `forkJoin` never emits if an input never completes. If an input completes without emitting a value, `forkJoin` completes without producing a combined result.

12. How do you handle errors in RxJS using catchError, retry and retryWhen?

Interview answer

RxJS provides operators to recover from errors, retry transient failures, and implement fallback behavior.

1. catchError

TypeScript

```
this.http.get<Employee[]>('/api/employees')
  .pipe(
    catchError(error => {
      console.error(error);
      return of([]);
    })
  )
  .subscribe(console.log);
```

`catchError` replaces a failed stream with another Observable.

2. retry

TypeScript

```
this.http.get<Employee[]>('/api/employees')
  .pipe(
    retry({
      count: 3,
      delay: 1000
    })
  )
  .subscribe(console.log);
```

Retries a failed request up to three times, with a one-second delay between attempts.

3. retryWhen

A legacy pattern for customized retry behavior:

TypeScript

```
this.http.get('/api/employees').pipe(
  retryWhen(errors =>
    errors.pipe(
      scan((attempt, error) => {
        if (attempt >= 3) {
          throw error;
        }
        return attempt + 1;
      }, 0),
      delay(1000)
    )
  )
)
```

```
)  
);
```

For modern RxJS, prefer the configurable `retry` operator for most retry policies.

Production-ready retry with exponential backoff

TypeScript

```
this.http.get<Employee[]>('/api/employees')  
  .pipe(  
    retry({  
      count: 3,  
      delay: (error, retryCount) => {  
  
        // Retry network failures and 5xx responses  
        const retryable =  
          error.status === 0 ||  
          error.status >= 500;  
  
        if (!retryable) {  
          return throwError(() => error);  
        }  
  
        const delayMs =  
          Math.min(1000 * 2 ** (retryCount - 1), 8000);  
  
        return timer(delayMs);  
      }  
    }),  
  
    catchError(error => {  
      console.error('Request failed:', error);  
      return throwError(() => error);  
    })  
  );
```

Senior-level considerations: Avoid blindly retrying non-idempotent POST operations, authentication failures or validation errors. Consider jitter, server-provided `Retry-After` values and retry limits for production systems.

13. What is the difference between `debounceTime`, `throttleTime`, `auditTime` and `sampleTime`?

Interview answer

These operators control how frequently values are emitted from a high-frequency source.

Operator	Behavior	Typical use
<code>debounceTime</code>	Emits after a quiet period	Search input
<code>throttleTime</code>	Emits first value, then suppresses values for a duration by default	Rate-limit clicks
<code>auditTime</code>	Emits the latest value at the end of a window	Scroll updates
<code>sampleTime</code>	Samples the latest value at periodic intervals	Monitoring

debounceTime

TypeScript

```
searchControl.valueChanges.pipe(  
  debounceTime(300)  
) .subscribe(search);
```

Waits until no new value has arrived for 300 milliseconds.

throttleTime

TypeScript

```
clicks$.pipe(  
  throttleTime(1000)  
) .subscribe(handleClick);
```

Emits the first click and suppresses subsequent clicks during the one-second window.

auditTime

TypeScript

```
scroll$.pipe(  
  auditTime(200)  
) .subscribe(updatePosition);
```

Emits the latest value at the end of each active audit window.

sampleTime

TypeScript

```
mouseMove$.pipe(  
  sampleTime(500)  
) .subscribe(updatePosition);
```

Samples the latest value every 500 milliseconds when a new value is available.

Interview tip: `debounceTime` waits for inactivity, while `auditTime` periodically emits the latest value after an active window. `sampleTime` follows a fixed sampling schedule.

14. How do you cancel an ongoing HTTP request when a component is destroyed?

Interview answer

Angular's `HttpClient` returns cancellable Observables. Unsubscribing from an in-progress HTTP request normally aborts the underlying client request.

Modern Angular provides `takeUntilDestroyed()`.

TypeScript

```
import { Component, DestroyRef, inject }  
  from '@angular/core';  
  
import { takeUntilDestroyed }  
  from '@angular/core/rxjs-interop';  
  
@Component({  
  selector: 'app-employee',  
  template: ''  
})  
export class EmployeeComponent {  
  
  private http = inject(HttpClient);  
  private destroyRef = inject(DestroyRef);  
  
  ngOnInit(): void {  
    this.http.get('/api/employees')  
      .pipe(  
        takeUntilDestroyed(this.destroyRef)  
      )  
      .subscribe({  
        next: data => console.log(data),  
        error: error => console.error(error)  
      });  
  }  
}
```

Alternative: AsyncPipe

TypeScript

```
employees$ =  
  this.http.get<Employee[]>('/api/employees');
```

HTML

```
{{ employees$ | async | json }}
```

`AsyncPipe` automatically unsubscribes when its view is destroyed.

Important: If the HTTP Observable is shared through a cache using `shareReplay({ refCount: false })`, unsubscribing from one component does not necessarily cancel the shared upstream request.

15. How would you implement polling using RxJS?

Interview answer

Polling periodically requests updated data from a server.

A typical implementation uses `timer`, `switchMap` and lifecycle cancellation.

Example: Refresh order status every five seconds

TypeScript

```
orderStatus$ = timer(0, 5000).pipe(  
  switchMap(() =>  
    this.http.get<OrderStatus>(  
      '/api/orders/1001/status'  
    )  
  ),  
  
  catchError(error => {  
    console.error(error);  
    return EMPTY;  
  } ),  
  
  takeUntilDestroyed(this.destroyRef)  
);
```

`timer(0, 5000)` triggers immediately and then every five seconds.

Alternative: Avoid overlapping requests

If a request can take longer than the polling interval, `switchMap` cancels the previous request.

Use `exhaustMap` when you want to let an active request finish and ignore polling ticks during its execution.

TypeScript

```
orderStatus$ = timer(0, 5000).pipe(
  exhaustMap(() =>
    this.http.get<OrderStatus>(
      '/api/orders/1001/status'
    ).pipe(
      catchError(error => {
        console.error(error);
        return EMPTY;
      })
    )
  ),
  takeUntilDestroyed(this.destroyRef)
);
```

Production considerations: Stop polling when the component is destroyed, when the operation reaches a terminal state, or when updates are no longer needed. Consider visibility-aware polling, exponential backoff and WebSockets for real-time use cases.

16. How do you handle dependent API calls and parallel API calls using RxJS?

Interview answer

Use `switchMap` or `concatMap` for dependent API calls and `forkJoin` for independent parallel HTTP calls.

Scenario 1: Dependent API calls

Fetch an employee, then fetch their department using the employee's department ID.

TypeScript

```
this.http.get<Employee>('/api/employees/1')
  .pipe(
    switchMap(employee =>
      this.http.get<Department>(
        `/api/departments/${employee.departmentId}`
      )
    )
  )
  .subscribe(department => {
    console.log(department);
  });
```

The second API depends on the response from the first API.

Preserve both responses

TypeScript

```
this.http.get<Employee>('/api/employees/1')
  .pipe(
    switchMap(employee =>
      this.http.get<Department>(
        `/api/departments/${employee.departmentId}`
      ).pipe(
        map(department => ({
          employee,
          department
        }))
      )
    )
  )
  .subscribe(result => {
    console.log(result.employee);
    console.log(result.department);
  });
```

Scenario 2: Parallel API calls

TypeScript

```
forkJoin({
  employees:
    this.http.get<Employee[]>('/api/employees'),

  departments:
    this.http.get<Department[]>('/api/departments'),
```

```
locations:
  this.http.get<Location[]>('/api/locations')
}).subscribe(result => {
  console.log(result);
});
```

All three requests can execute concurrently.

Interview tip: `forkJoin` is appropriate when all requests must finish before processing the combined result. If one request fails, the combined stream errors unless the failure is handled.

17. What is the difference between map, tap, filter and scan?

Interview answer

These operators serve different purposes in an Observable pipeline.

Operator	Purpose	Changes output?
<code>map</code>	Transforms each value	Yes
<code>tap</code>	Performs side effects	No
<code>filter</code>	Allows values matching a condition	Removes some values
<code>scan</code>	Accumulates state over time	Yes

map

TypeScript

```
of(1, 2, 3).pipe(
  map(x => x * 10)
).subscribe(console.log);

// 10, 20, 30
```

tap

TypeScript

```
of(1, 2, 3).pipe(  
  tap(x => console.log('Before:', x)),  
  map(x => x * 10)  
) .subscribe(console.log);
```

Useful for logging, debugging and non-transforming side effects.

filter

TypeScript

```
of(1, 2, 3, 4, 5).pipe(  
  filter(x => x % 2 === 0)  
) .subscribe(console.log);
```

```
// 2, 4
```

scan

TypeScript

```
of(1, 2, 3, 4).pipe(  
  scan((total, value) => total + value, 0)  
) .subscribe(console.log);
```

```
// 1, 3, 6, 10
```

`scan` is similar to `reduce`, but emits intermediate accumulated results rather than only the final result.

Senior interview tip: Avoid modifying application state inside `map`. Use `map` for pure transformations, `tap` for intentional side effects and `scan` for stream-based state accumulation.

18. How would you implement a centralized loading indicator using RxJS?

Interview answer

In a large Angular application, multiple HTTP requests may execute simultaneously. A simple Boolean loading flag can produce incorrect results because the first completed request may hide the spinner while other requests are still running.

A better approach is to maintain an active-request counter using `BehaviorSubject`.

Step 1: Loading service

TypeScript

```
@Injectable({ providedIn: 'root' })
export class LoadingService {

  private activeRequests = 0;

  private loadingSubject =
    new BehaviorSubject<boolean>(false);

  readonly loading$ =
    this.loadingSubject.asObservable();

  show(): void {
    this.activeRequests++;
    this.loadingSubject.next(true);
  }

  hide(): void {
    this.activeRequests = Math.max(
      0,
      this.activeRequests - 1
    );

    this.loadingSubject.next(
      this.activeRequests > 0
    );
  }
}
```

Step 2: HTTP interceptor

TypeScript

```
import {
  HttpInterceptorFn
} from '@angular/common/http';

import { inject } from '@angular/core';
import { finalize } from 'rxjs';

export const loadingInterceptor: HttpInterceptorFn =
  (req, next) => {

    const loadingService = inject(LoadingService);

    loadingService.show();
```

```

    return next(req).pipe(
      finalize(() => {
        loadingService.hide();
      })
    );
  };
};

```

Step 3: Register interceptor

TypeScript

```

export const appConfig: ApplicationConfig = {
  providers: [
    provideHttpClient(
      withInterceptors([loadingInterceptor])
    )
  ]
};

```

Step 4: Display loading indicator

TypeScript

```

export class AppComponent {
  loading$ = inject(LoadingService).loading$;
}

```

HTML

```

@if (loading$ | async) {
  <div class="spinner-border"
    role="status">
    <span class="visually-hidden">
      Loading...
    </span>
  </div>
}

```

How does it handle concurrent requests?

Request A starts

Active requests: 1

Spinner visible

Request B starts

Active requests: 2

Spinner visible

Request A completes

Active requests: 1

Spinner visible

Request B completes

Active requests: 0

Spinner hidden

Senior-level considerations: Use `finalize()` because it executes on completion, error and unsubscription. Provide an `HttpContextToken` to exclude background polling or silent refresh requests from the global spinner. A minimum display duration can also reduce flickering.

19. How do you test asynchronous RxJS streams using marble testing?

Interview answer

Marble testing uses virtual time to test asynchronous Observable behavior without waiting for real timers.

RxJS provides `TestScheduler` for deterministic testing.

Marble notation

Symbol	Meaning
-	Passage of virtual time
a, b, c	Emitted values
	Completion
#	Error
()	Simultaneous emissions
^	Subscription point
!	Unsubscription point

Example: Testing map

TypeScript

```
import { TestScheduler } from 'rxjs/testing';
import { map } from 'rxjs';
```

```

describe('RxJS map', () => {

  let scheduler: TestScheduler;

  beforeEach(() => {
    scheduler = new TestScheduler(
      (actual, expected) => {
        expect(actual).toEqual(expected);
      }
    );
  });

  it('should multiply values by 10', () => {

    scheduler.run(({ cold, expectObservable }) => {

      const source$ = cold('-a-b-c|', {
        a: 1,
        b: 2,
        c: 3
      });

      const result$ = source$.pipe(
        map(value => value * 10)
      );

      expectObservable(result$).toBe('-a-b-c|', {
        a: 10,
        b: 20,
        c: 30
      });
    });
  });
});

```

Example: Testing debounceTime

TypeScript

```

it('should debounce values', () => {

  scheduler.run(({ cold, expectObservable }) => {

    const source$ = cold(
      'a 100ms b 400ms |'
    );

    const result$ = source$.pipe(
      debounceTime(200)
    );
  });
});

```

```
expectObservable(result$).toBe(  
  '302ms b 200ms |'  
);  
});  
});
```

The first value is discarded because the second value arrives before its debounce window expires. The second value is emitted after 200 milliseconds.

What should senior developers test?

- Debouncing and throttling behavior.
- Cancellation using `switchMap`.
- Ordering with `concatMap`.
- Error recovery and retry timing.
- Subscription and unsubscription behavior.
- Shared Observable execution.

Interview tip: Marble tests verify stream timing and operator behavior. Use Angular's `HttpTestingController` for HTTP request and response testing. The two approaches complement each other.

20. How would you design a production-ready RxJS data service with caching, refresh, error handling and cancellation?

Senior architecture question

Interview answer

I would design a singleton Angular service that separates data fetching, caching, refresh triggers and error handling.

The service should expose a read-only Observable to components and provide explicit methods for refreshing and invalidating cached data.

For a more advanced application, I would also consider cache expiration, request deduplication, cancellation, observability and data consistency.

Architecture

Angular components

AsyncPipe / takeUntilDestroyed



RxJS Data Service

Refresh trigger · Cache · Error handling

switchMap + shareReplay(1)



Angular HttpClient

HTTP interceptor



Backend REST API

Step 1: Define the model

TypeScript

```
export interface Employee {  
  id: number;  
  name: string;  
  departmentId: number;  
}
```

Step 2: Implement the service

This example uses a reactive refresh trigger, shared responses, retry handling and automatic recovery after request failures.

TypeScript

```
import { Injectable, inject } from '@angular/core';  
import { HttpClient } from '@angular/common/http';  
  
import {  
  BehaviorSubject,  
  Observable,  
  catchError,  
  retry,  
  shareReplay,  
  switchMap,  
  throwError,  
  timer  
}
```

```

} from 'rxjs';

@Injectable({
  providedIn: 'root'
})
export class EmployeeDataService {

  private readonly http = inject(HttpClient);

  private readonly apiUrl = '/api/employees';

  private readonly refreshTrigger$ =
    new BehaviorSubject<void>(undefined);

  private readonly errorSubject =
    new BehaviorSubject<string | null>(null);

  readonly error$ =
    this.errorSubject.asObservable();

  readonly employees$: Observable<Employee[]> =
    this.refreshTrigger$.pipe(

      switchMap(() => {

        this.errorSubject.next(null);

        return this.http.get<Employee[]>(
          this.apiUrl
        ).pipe(

          retry({
            count: 2,
            delay: (error, attempt) => {

              if (
                error.status !== 0 &&
                error.status < 500
              ) {
                return throwError(() => error);
              }

              return timer(1000 * attempt);
            }
          )),

          catchError(error => {

            this.errorSubject.next(
              'Unable to load employees'
            );
          })
        );
      })
    );

```

```

        return throwError(() => error);
    })
);
}),

shareReplay({
    bufferSize: 1,
    refCount: true
})
);

refresh(): void {
    this.refreshTrigger$.next();
}
}

```

Why this design?

`BehaviorSubject` provides the refresh trigger and immediately starts the first request when the stream is subscribed to.

`switchMap` cancels an earlier request when a new refresh occurs.

`shareReplay(1)` shares the request and its latest result among active subscribers.

`retry` handles transient failures, while `catchError` updates error state and propagates the error.

Important limitation: An unhandled error terminates `employees$`. For a resilient long-lived data stream, return a typed result or state object from the inner `catchError` instead of rethrowing. That allows future refresh triggers to continue working.

Also, with `refCount: true`, the cache is not guaranteed to remain populated when all subscribers disappear before the HTTP request completes.

Step 3: More resilient production implementation

A typed state model lets the UI represent loading, success and failure without terminating the refresh stream.

TypeScript

```

export interface DataState<T> {
    data: T | null;
    loading: boolean;
}

```

```
error: string | null;
```

TypeScript

```
import {
  BehaviorSubject,
  catchError,
  map,
  of,
  shareReplay,
  startWith,
  switchMap
} from 'rxjs';

@Injectable({ providedIn: 'root' })
export class EmployeeStore {

  private http = inject(HttpClient);

  private refresh$ =
    new BehaviorSubject<void>(undefined);

  readonly state$ = this.refresh$.pipe(

    switchMap(() =>
      this.http.get<Employee[]>('/api/employees').pipe(

        map(employees => ({
          data: employees,
          loading: false,
          error: null
        }) as DataState<Employee[]>)),

        startWith({
          data: null,
          loading: true,
          error: null
        }) as DataState<Employee[]>),

        catchError(() =>
          of({
            data: null,
            loading: false,
            error: 'Failed to load employees'
          }) as DataState<Employee[]>
        )
      )
    ),

    shareReplay({
      bufferSize: 1,
      refCount: true
    })
  )
}
```

```

    })
  );

  refresh(): void {
    this.refresh$.next();
  }
}

```

This implementation keeps the outer refresh stream alive after individual HTTP failures.

Step 4: Component implementation

TypeScript

```

@Component({
  selector: 'app-employees',
  templateUrl: './employees.component.html'
})
export class EmployeesComponent {

  private store = inject(EmployeeStore);

  state$ = this.store.state$;

  refresh(): void {
    this.store.refresh();
  }
}

```

Step 5: Angular template

HTML

```

@if (state$ | async; as state) {

  @if (state.loading) {
    <p>Loading employees...</p>
  }

  @if (state.error) {
    <p class="text-danger">
      {{ state.error }}
    </p>
  }

  @if (state.data; as employees) {
    @for (employee of employees; track employee.id) {
      <p>{{ employee.name }}</p>
    }
  }
}

```

}

Production improvements

For an enterprise application, extend the design with:

- **TTL-based caching:** Expire responses after a configurable period.
- **Cache invalidation:** Invalidate affected employee data after create, update or delete operations.
- **Stale-while-revalidate:** Keep previously loaded data visible while fetching a fresh response.
- **Request deduplication:** Share in-flight requests for identical resource keys.
- **Cancellation:** Use `switchMap` and component lifecycle cleanup.
- **Error recovery:** Keep refresh streams alive and expose errors separately from data.
- **Testing:** Verify concurrent subscriptions, refresh behavior, retries and cancellation.

Senior interview: Quick revision cheat sheet

Choose the right RxJS operator

Tap each scenario to reveal the recommended operator.

1 Cancel an outdated search request ^

`switchMap`

Keeps only the latest inner subscription.

2 Run independent requests concurrently ^

`mergeMap`

Allows multiple inner subscriptions at the same time.

3 Save changes in strict order ^

`concatMap`

Queues requests and executes them sequentially.

4 Ignore repeated clicks during submission ^

`exhaustMap`

Ignores new emissions while the current operation is active.

5 Cache a successful HTTP response ^

`shareReplay(1)`

Shares execution and replays the latest response.

6 Wait for several HTTP calls to finish ^

`forkJoin`

Emits the final results after every input completes.

7 Automatically clean up component subscriptions ^

`takeUntilDestroyed`

Unsubscribes when the Angular destroy context is destroyed.

8 Accumulate values over time ^

`scan`

Emits each intermediate accumulated result.

9 Execute cleanup on error, completion or cancellation ^

`finalize`

Runs teardown logic for all three termination paths.

10 Recover from a failed HTTP request ^

`catchError`

Replaces an error with another Observable.

Practice: Test your RxJS interview knowledge

1. Which operator should you use for autocomplete to cancel outdated requests?

- ☐ mergeMap
- ☐ switchMap
- ☐ concatMap
- ☐ exhaustMap

2. Which Subject immediately emits its current value to a new subscriber?

- ☐ Subject
- ☐ AsyncSubject
- ☐ BehaviorSubject
- ☐ ReplaySubject

3. Which operator waits for all HTTP Observables to complete?

- ☐ combineLatest
- ☐ forkJoin
- ☐ withLatestFrom
- ☐ share

4. Which operator prevents overlapping login requests by ignoring additional clicks?

- ☐ switchMap
- ☐ mergeMap
- ☐ exhaustMap
- ☐ concatMap

5. Which operator is used for cleanup on completion, error and unsubscription?

- ☐ map
- ☐ filter

☐ finalize

☐ scan

6. What does `shareReplay(1)` cache?

☐ All historical values

☐ The latest emitted value

☐ Only errors

☐ Nothing

7. Which operator preserves the order of asynchronous `save` requests?

☐ `mergeMap`

☐ `switchMap`

☐ `concatMap`

☐ `exhaustMap`

8. Which Angular utility handles subscription cleanup on component destruction?

☐ `takeUntilDestroyed`

☐ `retry`

☐ `shareReplay`

☐ `forkJoin`

Submit answers

Final interview preparation advice: For senior Angular interviews, focus on explaining *why* you choose an operator, what happens under concurrent requests, how subscriptions are cleaned up, and how your implementation behaves when an API fails or a user refreshes data. These architectural considerations are often more important than simply memorizing operator definitions.

Is this conversation helpful so far?

